EXP5

April 10, 2026

[2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, auc, classification_report
from sklearn.preprocessing import StandardScaler
```

[4]:
```
data = pd.read_csv("diabetes.csv")

print(data.head())
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

[6]:
```
X = data.drop('Outcome', axis=1)
y = data['Outcome']
```

[8]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[10]:
```
model_none = LogisticRegression(penalty=None, max_iter=1000)
model_none.fit(X_train, y_train)
```

[10]:
```
LogisticRegression(max_iter=1000, penalty=None)
```

[12]:
```
model_l1 = LogisticRegression(penalty='l1', solver='liblinear', C=1.0)
model_l2 = LogisticRegression(penalty='l2', solver='lbfgs', C=1.0)

model_l1.fit(X_train, y_train)
model_l2.fit(X_train, y_train)
```

[12]:
```
LogisticRegression()
```

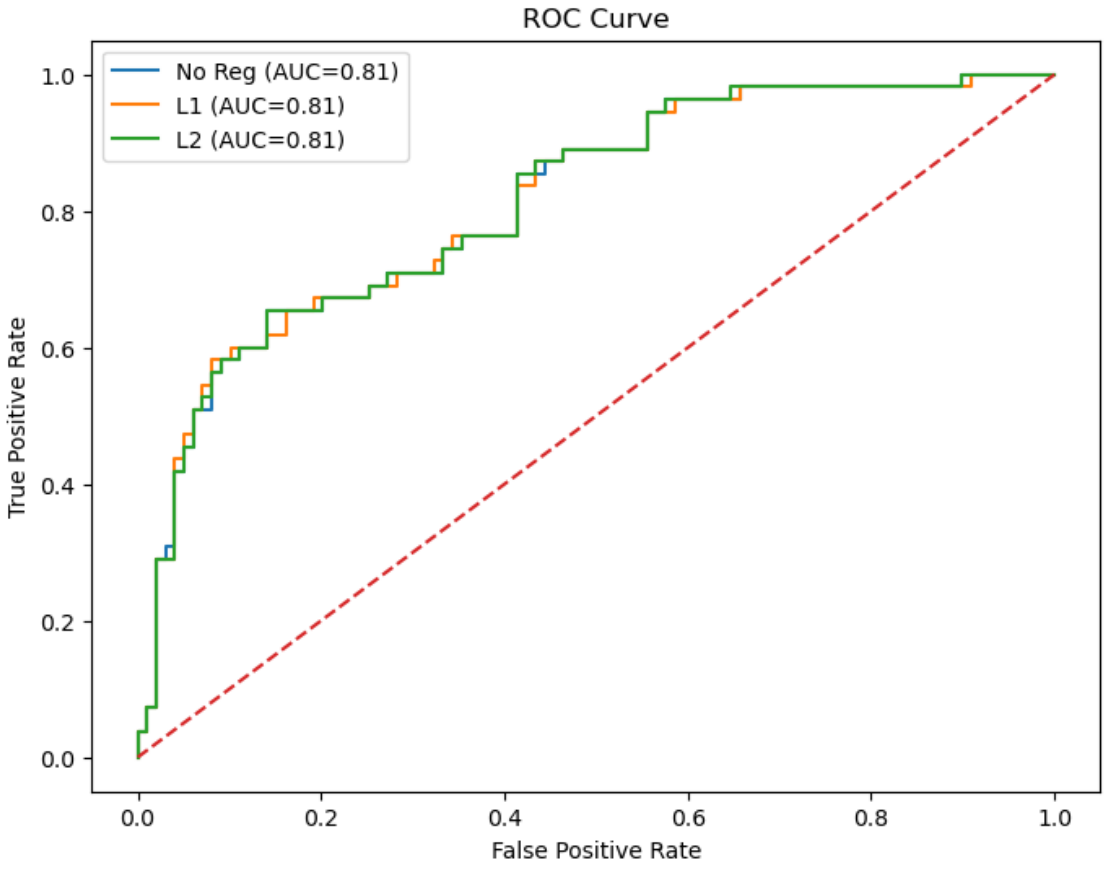
[14]:
```
models = {
    "No Reg": model_none,
    "L1": model_l1,
    "L2": model_l2
}

plt.figure(figsize=(8,6))

for name, model in models.items():
    y_prob = model.predict_proba(X_test)[:,1]
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    roc_auc = auc(fpr, tpr)

    plt.plot(fpr, tpr, label=f"{name} (AUC={roc_auc:.2f})")

plt.plot([0,1], [0,1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```



[16]:
```
for name, model in models.items():
    y_pred = model.predict(X_test)
    print(f"\n{name} Model:")
    print(classification_report(y_test, y_pred))
```

```
No Reg Model:
              precision    recall  f1-score   support

           0       0.81      0.80      0.81        99
           1       0.65      0.67      0.66        55

    accuracy                           0.75       154
   macro avg       0.73      0.74      0.73       154
weighted avg       0.76      0.75      0.75       154


L1 Model:
              precision    recall  f1-score   support
```

```
              0       0.81      0.79      0.80        99
              1       0.64      0.67      0.65        55

       accuracy                           0.75       154
      macro avg       0.73      0.73      0.73       154
   weighted avg       0.75      0.75      0.75       154


L2 Model:
              precision    recall  f1-score   support

              0       0.81      0.80      0.81        99
              1       0.65      0.67      0.66        55

       accuracy                           0.75       154
      macro avg       0.73      0.74      0.73       154
   weighted avg       0.76      0.75      0.75       154
```

[18]:
```
param_grid = {'C': [0.01, 0.1, 1, 10, 100]}

grid = GridSearchCV(
    LogisticRegression(penalty='l2', solver='lbfgs', max_iter=1000),
    param_grid,
    cv=5,
    scoring='f1'
)

grid.fit(X_train, y_train)

print("Best C:", grid.best_params_)
print("Best Score:", grid.best_score_)
```

```
Best C: {'C': 10}
Best Score: 0.6240550480728033
```

[ ]:
```

```